

5/5 Questions Answered

Saved at 8:44 PM

Pset 4 - Comprehension

Q1

2 Points

Recall that in lecture 7, we explored a way to encode objects using a `lambda` that referred to boxes storing the fields of the object. In that encoding, the `lambda` for the object took an argument `m` representing which method to call, and did a `case` on `m`, returning a function for the corresponding method. To support self-reference in objects, each of our methods took as their first argument the object itself. To simplify invoking a method on an object and handling the task of passing the object as the first argument, we wrote a helper function called `msg/self` defined as:

```
(define (msg/self o m . a)
  (apply (o m) o a))
```

With this definition, the following example will return 7, as intended:

```
(define o-self
  (lambda (m)
    (case m
      [(first) (lambda (self x) (msg/self self 'second (+ x 1)))]
      [(second) (lambda (self x) (+ x 1))]))

(msg/self o-self 'first 5)
```

Imagine that instead of implementing `msg/self` as a function in the above, one instead tried to use the following macro definition:

```
(define-syntax (msg/self-2 stx)
  (syntax-case stx ()
    [(msg/self-2 o m a ...)
     #'((o m) o a ...)]))
```

Which of the following is true:

- ☐ `msg/self-2` is a valid alternative definition of `msg/self` and produces the same behaviors as `msg/self`.
- ☐ `msg/self-2` is incorrect, and can differ from `msg/self` when used in an example of the form `(msg/self-2 e1 m e2)` when evaluating the expression `e2` modifies mutable state, even if evaluating `e1` does not modify mutable state.
- ☒ `msg/self-2` is incorrect, and can differ from `msg/self` when used in an example of the form `(msg/self-2 e1 m e2)` when evaluating the expression `e1` modifies mutable state, even if evaluating `e2` does not modify mutable state.

Save Answer

Last saved on Nov 21 at 8:39 PM

Q2

6 Points

In class, we informally used some notation when trying to describe the continuation captured by `let/cc`. For example, in a snippet of code like

```
(+ 1 (+ 2 (+ 3 (+ (let/cc k 4) 5))))
```

we say that the continuation bound to `k` looked like

```
(+ 1 (+ 2 (+ 3 (+ [...] 5))))
```

where if we were to apply `k`, as in `(k v)` the `[...]` represents a "hole" into which the value `v` will be "plugged-in".

Q2.1

2 Points

Using the notation in the above example, which of the following represents the continuation bound to `k` in `let/cc` when executing `(+ (+ 3 5) (+ (let/cc k 4) (+ 6 11)))`

- ☐ `(+ 8 (+ (+ 4 [...]) (+ 6 11)))`
- ☐ `(+ 8 (+ [...] 17))`
- ☒ `(+ (+ 3 5) (+ [...] (+ 6 11)))`
- ☐ `(+ 8 (+ [...] (+ 6 11)))`

Save Answer

Last saved on Nov 21 at 8:44 PM

Q2.2

2 Points

Using the notation in the above example, which of the following represents the continuation bound to `k` in `let/cc` when executing

```
((lambda (x) (+ (+ x 2) (+ (let/cc k 1) (+ 1 3)))) (+ 2 2))
```

- ☐ `((lambda (x) (+ (+ x 2) (+ [...] 4))) (+ 2 2))`
- ☒ `(+ 6 (+ [...] (+ 1 3)))`
- ☐ `((lambda (x) (+ (+ x 2) (+ [...] (+ 1 3)))) (+ 2 2))`
- ☐ `(+ 6 (+ [...] 4))`

Save Answer

Last saved on Nov 21 at 8:44 PM

Q2.3

2 Points

Using the notation in the above example, which of the following represents the continuation bound to `k` in `let/cc` when executing

```
((lambda (f) (+ (+ 1 2) (+ (f 1) (+ 1 3)))) (lambda (x) (let/cc k 1)))
```

- ☐ `((lambda (f) (+ (+ 1 2) (+ (f 1) (+ 1 3)))) (lambda (x) [...]))`
- ☐ `(+ (+ 1 2) (+ ([...] 1) (+ 1 3)))`
- ☐ `(+ 3 (+ [...] 4))`
- ☒ `(+ 3 (+ [...] (+ 1 3)))`

Save Answer

Last saved on Nov 21 at 8:44 PM

Q3

2 Points

Consider the following Racket program, which defines two macros called `mlt-a` and `mlt-b`.

```
(define-syntax (mlt-a stx)
  (syntax-case stx ()
    [(mlt-a ([var1 val1] [var2 val2]) body)
     #'((lambda (var1) ((lambda (var2) body) val2)) val1)]))

(define-syntax (mlt-b stx)
  (syntax-case stx ()
    [(mlt-b ([var1 val1] [var2 val2]) body)
     #'(((lambda (var1) (lambda (var2) body)) val1) val2)]))
```

Which of the following best describes these two macros?

- ☐ They both implement behavior that is similar to Racket's `let*` but requires defining exactly two variables at a time.
- ☒ `mlt-a` behaves like Racket's `let*` and `mlt-b` behaves like Racket's `let`, except for defining exactly two variables at a time.
- ☐ They both implement behavior that is similar to Racket's `let` but requires defining exactly two variables at a time.
- ☐ `mlt-a` behaves like Racket's `let` and `mlt-b` behaves like Racket's `let*`, except for defining exactly two variables at a time.

Save Answer

Last saved on Nov 21 at 8:44 PM

Save All Answers

Submit & View Submission >